

Fuzz testing a Discrete Event Simulator

a code walk-thru

Apoorv Kothari
toidiu.com

Overview

- Architecture layers
 - SansIO: simplify tests via determinism
 - Discrete Event Simulator: accelerate testing
 - Fuzz: increase testing surface area
 - AI: parallelize reasoning
- Code walk-thru / Demo

Architecture layers

Use of testing methodologies

- Architecture found in software
 - Unit (~99%)
 - Integration (75-65%)
 - Deterministic code (SansIO) (45-25%)
 - Discrete Event Simulator (1-5%)
 - Fuzz (5-10%)
 - Fuzz driven Discrete Event Simulator (~1%)

% based on author's hunch

Unit / Integration tests

Pros

- Easy to write and execute

Cons

- Limited to the author's imagination and effort (AI can help here)
 - Can be difficult to write
- Results in testing-code bloat

Deterministic Code (SansIO *)

Pros

- Code is easier to write and reason about
 - Easier to onboard new contributors
- Bugs become easier to reproduce
 - Easier and faster to debug issues
- Fewer dependencies
 - Easier maintenance
 - Faster compile times

Cons

- Most of the complexity is shifted to the interface layer
- Lack of training material on how to do this well

* https://github.com/toidiu/raft-rs/blob/main/sansio_presentation.pdf

Discrete Event Simulator

Pros

- Leader election test (~280x Faster!! *)
 - Integration test: 300 / min (limited by timeout, packet delay, etc...)
 - DES: 85,000 tests / min (limited by CPU instructions/sec)

Cons

- Requires a deterministic codebase
- Needs a simulation environment
- Lack of training material on how to do this well

* This is a conservative value. The speedup is massive (multiple 1000x) if testing packet delay.

Fuzz

Pros

- Generates “smart” test variations automatically
- Reduces testing-code bloat

Neutral

- Less effective as problem space increases (“random” path exploration)

Cons

- Can be complicated to set up
- Lack of training material on how to do this well

AI

Pros

- Great at reasoning about small scope

Cons

- Poor at reasoning about large scope
 - Works best when given concrete success/failure criteria
- Lack of training material on how to do this well

Code walk-thru / Demo

Code walkthrough

- Raft high level: <https://raft.github.io/>
- Sim: <https://github.com/toidiu/raft-rs/commit/16ec84d0089e21cd22de0663bc0dbfe3fd992583>
 - Cluster, Network, Sim (MAX_EVENTS), Inspect, Failure
 - What does a test look like: [Replication test](#), [Inspect server state](#)
- Fuzz: <https://github.com/toidiu/raft-rs/commit/a5f0750fe29b7e743122655e0fc5fc3c44d2c5dd>
 - Operation, fuzz_raft, make fuzz

References

- raft-rs: <https://github.com/toidiu/raft-rs/>
- sansIO: https://github.com/toidiu/raft-rs/blob/main/sansio_presentation.pdf
- Fuzz testing: https://antithesis.com/docs/resources/property_based_testing

Fin

Apoorv Kothari

toidiu.com

afterword

AI is the fast reasoning layer in our tech stack.

Similar to how adding more members to a project doesn't accelerate delivery, AI by itself doesn't accelerate delivery. We need to intentionally “onboard” it, provide guard rails and design systems with AI in mind.

AI is most effective when the reasoning task is scoped and small.

Our jobs as software engineers prior to AI was to design systems that were easy to reason about (KISS: keep it simple stupid). Armed with a faster reasoning engine, the job has shifted to designing systems that enable fast but scoped reasoning.